

1. MVC 與三層架構的關係

- **View (視圖層)**：對應展示層，負責 UI 呈現，讓使用者看到結果。
- **Controller (控制器)**：對應業務邏輯層，負責處理邏輯、流程控制。
- **Model (模型層)**：對應資料層，負責資料的存取、管理與更新。

差異點：MVC 是設計模式，硬體三層式架構則是系統架構概念。前者偏重應用邏輯分離，後者注重層級間職責明確。

2. v-if 和 v-for 優先順序

- **v-if** 先執行，之後才會執行 **v-for**。
- 避免同時使用，會造成效能浪費或預期外的行為。

推薦做法：用 `<template>` 包裹 `v-for`，再內部使用 `v-if`。

差異點：

- `v-if`：條件為 `false` 時，元素不會渲染。
 - `v-show`：元素會保留在 DOM 中，只是設為 `display: none`。
-

3. 子元件傳遞資料至父元件

- **\$emit**：使用自定義事件，讓子元件通知父元件資料變化。適合單次傳遞。
- **v-model**：雙向綁定，透過 `modelValue` 和 `update:modelValue` 實現，適合雙向資料同步。

差異點：

- `$emit`：主要用於事件傳遞。
 - `v-model`：用於資料雙向同步，適合輸入框或即時更新場景。
-

4. Vue 3 如何異動 ref 物件資料

在 Vue 3 中，`ref` 用來創建一個響應式的參考，並且通常用來引用 DOM 元素或儲存基本資料型別。若要異動 `ref` 物件的資料，可以直接修改物件的屬性。

5. watch 和 computed 的差異與使用時機

- **computed**：

- 適合依賴多個資料進行計算的場合。
- 具備快取機制，依賴不變時不會重新計算。

- **watch** :

- 監聽資料變化，適合執行邏輯或非同步操作。
- 可監聽多個資料、設定 `immediate` 。

使用時機：

- `computed` : 用於展示計算結果，提升效能。
 - `watch` : 用於監聽變化並觸發副作用，如 API 請求。
-

6. `computed` 和 `method` 的差異與使用時機

- **computed** :

- 具快取特性，只有依賴變更時才重新執行。
- 適合多次重複使用的計算結果。

- **method** :

- 每次呼叫都會重新執行，不具快取。
- 適合動態傳遞參數或即時計算。

使用時機：

- `computed` : 適合需要重用且依賴變化的資料。
 - `method` : 適合單次即時運算、不需要快取的場合。
-

7. `computed` 的快取特性

- **快取** : `computed` 會根據響應式依賴進行快取。
- 當依賴的資料未變化時，會返回先前結果而不重新計算。

優勢：

- 減少重複計算，提升效能。
-

8. `watch` 的非同步操作支援

- `watch` 支援非同步邏輯，可用於 API 請求或異步任務。
- 設定 `{ immediate: true }` 可以在資料初始化時立刻觸發。

注意：需要透過 `try/catch` 處理非同步錯誤，避免影響程式執行。

9-10. 效能考量：computed vs method

- **computed**：適合需要快取的計算，依賴變化時才重新運算。
- **method**：適合不需要快取，或需要動態參數的運算邏輯。

優化建議：

- 頻繁執行的邏輯優先考慮 `computed`，減少重複計算。
- 單次運算或動態傳參則使用 `method`。

11. Vue 生命週期，舉出三個常用

1. **created**：

- 組件實例已創建，資料已初始化，但組件尚未掛載到 DOM。
- **無法操作 DOM**，只能初始化資料，進行數據設置或進行某些初始化操作。適合做初始的數據處理、API 請求等。

2. **mounted**：

- DOM 掛載完成後執行，適合進行 DOM 操作或 API 請求。
- **DOM 已存在**，可進行渲染後邏輯，如操作 DOM、發送請求、設定事件監聽等。

3. **updated**：

- 當資料變化並導致 DOM 更新後觸發。
- 當組件中的資料發生變化並且 DOM 更新後，這個鉤子會被觸發。適合在資料變動後進行額外操作，如調整頁面元素、進行動畫等。

12. ref 和 reactive 的差異

- **ref**：適合基本類型，需透過 `.value` 存取。
- **reactive**：適合物件或陣列，直接操作屬性觸發響應式。

比較：

- `ref`：適用於基本型別，適合傳遞至模板。
- `reactive`：適用於複雜資料結構，但不能解構後保留響應性。

13. v-once 的使用場景

- 渲染靜態內容或不會變動的內容，只渲染一次。
 - 適合大量靜態元素，提升效能。
-

14. var、let 和 const 的差異

- **var**：函數作用域，可重複宣告，存在變量提升。
- **let**：塊級作用域，不可重複宣告，適合可變數據。
- **const**：塊級作用域，不可重新賦值，適合常量或引用類型資料。

最佳實踐：

- **const**：預設使用，避免誤修改。
- **let**：當需要重新賦值時使用。
- **var**：僅用於舊代碼兼容，避免使用。